

Day 74

* Data Encapsulation and Data Abstraction:-* Data Encapsulation

- The process of hiding the confidential information / data / methods from external programmers / end users is called "Data Encapsulation".
- The purpose of Encapsulation concept is that "To hide confidential information / features of class (Data members and methods)".
- Data Encapsulation can be applied in two levels. They are:
 - a) At Data members level
 - b) At method level
- To implement Data Encapsulation in Python programming, the data members, methods must be preceded with under score (_).

Syntax 1). (Data member level through method)

```
class <className>:
    def methodname(self):
        self.__Data Member Name 1 = value 1
        self.__Data Member Name 2 = value 2
        —————
        self.__Data Member Name - n = value - n
```

(OR)

(Data member level through constructor).

```
class <className>:
    def __init__(self):
        self.__Data Member Name 1 = value 1
        self.__Data Member Name 2 = value 2
        —————
        self.__Data Member Name - n = value - n
```

Syntax 2). (Method level)

```
class <Class Name>:
    def __methodname(self):
        self.Data Member Name 1 = value 1
        self.Data Member Name 2 = value 2
        —————
        self.Data Member Name - n = value - n
```

* Data Abstraction:-

→ the process of retrieving / extracting essential details without considering hidden details is called Data Abstraction.

Note:- we can't apply Data Encapsulation or Constructors in Python but whose initialized data members can be encapsulated.

Note:- We can also encapsulate class name but in real time hiding the class name is of no use bcoz we get ImportError.

syntax class __ <classname> :-

Program for demonstrating Data Encapsulation

Account1.py - - - - - Filename & module name -- Data Encapsulation,

```
class Account:
    def __init__(self):
        self.__acno = 1234
        self.__cname = "Naseem"
        self.__bal = 3.4
        self.__pin = 8989
        self.__bname = "SBD"
```

~~# Account1.py~~ # others1.py - - Data Abstraction

```
from Account1 import Account
ac = Account()
# print("Account Number: {}".format(ac.acno))
# print("Account Holder name: {}".format(ac.cname))
# print("Account Balance: {}".format(ac.bal))
# print("Account pin: {}".format(ac.pin))
# print("Account Branch name: {}".format(ac.bname))
```

Account2.py.

By isme) def getaccdetails(self): बाकी सब सैंम

others2.py

```
[ from Account2
  ac = Account()
  ac.getaccdetails() ] बाकी सब सैंम
```

Account3.

sab same) By def __defaccdetails(self).

others3

→ ये दो रूँगा बाकी सब comment कर देना है

Account4

class __Account (बस यही change करना है, बाकी सब सैंम).

others4

others3 को sab comment out कर देना है

Constructor kiding.

Account 5.py

Bas. def __init__(self): ← वैसे इसी में 'underscore' लगाना है।
Bank's sb name में है

other 5.py

Bas isme. ac = Account() के बाद

ac.__init__() # method call;

बाकी सब वैन but comment out.

Same Account.py - - - file name & module name - - Data Encapsulation

class Account:

def __init__(self):

इसमें ex1 Account का वैन.

def __sameCustoms(self):

print(' - - - ')

print("Current customer details").

बाकी सब वैन ex1 other में print में पूरा

def show(self):

Account.__sameCustoms(self)

main program

ao = Account() # object creation calls default constructor

ao.show()

Q. write a python program which will accept student number,
A). student name and marks and save those details in a file
by using class object with pickling operation

B). write a python program which will read student records from the
file and display by using classes & objects. with unpickling
operation.

Day 57

* Inheritance :-

- Inheritance is one of distinct features of oops.
- The purpose of inheritance is that "to build re-usable Applications with effective memory management in python object oriented programming".

* Definition of Inheritance

- The process obtaining Data members, methods and Constructors (features) of one class into another class is called "Inheritance".
- The Class which is giving Data members, methods & Constructors (features) is called super or Base or parent class.
- The class which is taking Data members, methods & Constructors (features) is called sub or Derived or Child class.
- The inheritance Concept always follows logical or virtual memory management. This memory management says that 'Neither we write source code nor takes physical memory space'.

* Advantages

- ⇒ When we develop any inheritance based Application, we get the following Advantages:
 - 1) Application development time is less
 - 2) Application memory space is less
 - 3) Application execution time is fast/less
 - 4) Application performance is enhanced (improved)
 - 5) Redundancy (Duplication) of the code is minimized

* Intending the Features of Base Class into Derived Class

→ To inherit the features of Base Class into Derived Class, we use the following syntax:

```
class <classname-1>:
```

```
class <classname-2>:
```

```
class <classname-n>:
```

```
class <classname-n+1>(<classname-1, <classname-2, ... <classname-n>):
```

• classname-n+1,

↓ represents name of the derived or subclass.

• classname-1, classname-2, ... classname-n,

↓ represents name of the Base or super class.

→ Here All the Data members, methods and constructors are inherited from classname-1, classname-2, ... classname-n into classname-n+1 virtually or logically.

→ Here ~~derived~~ Derived class classname-n+1,

↓ can access the all the features of Base classes.

→ When we develop any inheritance Based Application, it is always recommended to create an object of Bottom most derived class bcz it inherits all the features of Base class, intermediate Base class (eg).

→ For every class in python, there exists an implicit pre-defined super class "object" and it provides Garbage collection facility.

* Types of Inheritance OR Re-usable Tech in Python:-

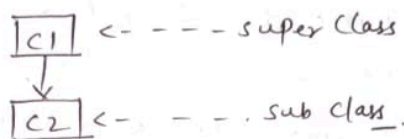
→ In Python programming, we have 5 types of inheritance. They are

- 1) Single inheritance
- 2) Multi level inheritance
- 3) Hierarchical inheritance
- 4) Multiple inheritance
- 5) Hybrid inheritance

1) Single inheritance:-

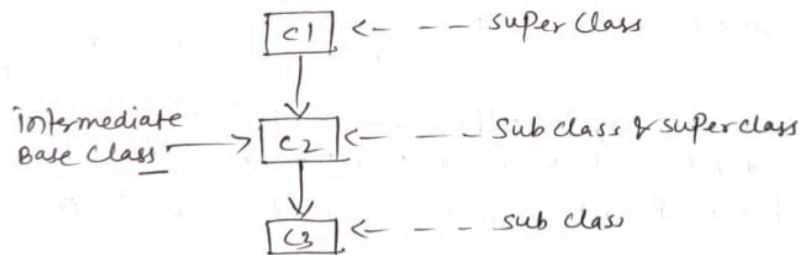
→ This inheritance contains Single Base class and single derived class.

Diagram



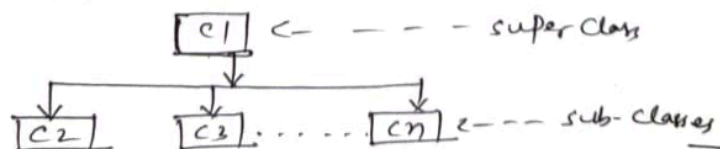
2) Multi level inheritance:-

→ This inheritance contains single base class, single derived class and intermediate base class(es).



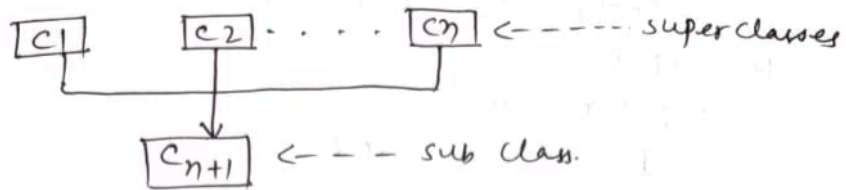
3) Hierarchical inheritance:-

→ This inheritance contains single super class and multiple sub classes.



4) multiple inheritance:-

→ This inheritance contains multiple superclasses and single sub class.



5) Hybrid inheritance:-

→ combination of any available inheritance types

Diagram 1)

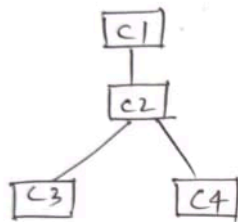
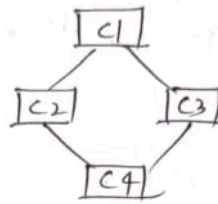


Diagram 2)



Alphabet 69

Program for demonstrating non-inheritance --- Class & objects

class C1:

```
def disp1(self):
    print("C1 ... disp1()")
```

class C2:

```
def disp2(self):
    print("C2 ... disp2()")
```

#main program

O1 = C1()

O2 = C2()

O1.disp1()

O2.disp2()

Program for demonstrating inheritance

ex1). class c1:
 def disp1(self):
 print("c1 ... disp1()")
 class c2(c1):
 def disp2(self):
 print("c2 ... disp2()")

main program

o2 = c2()
 o2.disp1()
 o2.disp2()

ex2). $\rightarrow$ B as def k niche self.disp1().
 aur main program me, disp1 nahi likhna hai.

ex3). class c1:
 def getA(self):
 self.a = 10
 class c2(c1):
 def getB(self):
 self.b = 20
 class c3(c2):
 def getC(self):
 self.c = 30
 def operation(self):
 self.d = self.a + self.b + self.c
 print("Sum({}, {}, {}) = {}".format(self.a, self.b, self.c, self.d))

main program

o3 = c3()
 print(o3.__dict__) # {}
 o3.getA()
 print(o3.__dict__) # {'a': 10}.
 o3.getB()
 print(o3.__dict__) # {'a': 10, 'b': 20}.
 o3.getC()
 print(o3.__dict__) #
 o3.operation()
 print(o3.__dict__) -

ex4) Bay def operation #). Change
 def operation(self):
 self.get A()
 self.get B()
 self.get C()
 self.d = self.a + self.b + self.c
 → Back print.

#main
 o3 = c3
 o3.operation

ex5) class C1 as it is ex3.
 c2 अकेला की है साथ में नहीं.
 c3.(c1, c2). बाकी सब सही.

1. write a python program which will implement the following
 → let us assume there exist a university which contains university name, location.
 accept and display university details
- let us assume there exist a college which contains college name and location accept and display college details along with university details
- let us assume there exist a student which contains student number, student name and course. accept and display student details along with college and university details.

→ Univ.py ← --- filename & module name

```

class university:
    def getunivdet(self):
        self.uname = input("Enter university name:")
        self.uloc = input("Enter university location:")

    def dispunivdet(self):
        print("-" * 50)
        print("university details")
        print("-" * 50)
        print("university name: {}".format(self.uname))
        print("university location: {}".format(self.uloc))
        print("-" * 50)
  
```

U
↓
C
↓
S.

```

# class 'College'
from univ import university
class College(university):
    def getColldet(self):
        self.cname = input("Enter College name")
        self.cloc = input("College location")
        self.getunivdet() # Calling Base Class method from
                           # derived class method.
    def dispColldet(self):
        self.dispunivdet() # Calling Base Class method
                           # from derived class method
        print("-" * 50)
        print("College details")
        print("-" * 50)
        print("College name: {}".format(self.cname))
        print("College location: {}".format(self.cloc))
        print("-" * 50)

```

```

class student(College):
    def getstudet(self):
        self.sno = int(input("Student number"))
        self.sname = input("Student name")
        cns = input("CNS") → self.getColldet()
                           self.getunivdet()
    def dispstudet(self):
        print("-" * 50)
        print("\t Student Details")
        print("-" * 50)
        print("\t Student number: {}".format(self.sno))
        print("\t Student name: {}".format(self.sname))
        print("\t CNS: {}".format(self.cns))
        print("-" * 50)

```

selfdisp ←
univdet()
selfdispColldet

```

# main program
s0 = student()
s0.getstudet()
s0.getColldet()
s0.getunivdet()
s0.dispunivdet()
s0.dispColldet()
s0.dispstudet()

```

```

# student.py
from college import College.

# univ college student.py.
from student import student.
main program
↓
s = student()
s.getstudent()
s.displaystudent()

```

Day 76

* polymorphism in python:-

- Polymorphism is one of the distinct features of oops.
- the purpose of polymorphism is that "Efficient utilization of memory space (or) less memory space is achieved."

Definition of polymorphism:-

- the process of representing "one form in multiple forms" is called "polymorphism".
- the polymorphism principle is implemented (bring into action) by using "method overriding" features of all programming languages.
- In the definition of polymorphism, "one form" represents "original method" and multiple forms represents overridden methods.
- A "Form" is nothing but existence of a method. if the method is existing in base class then it is called "original method (one form)" and if the method existing in derived class(es) then it is called "overridden method (multiple forms)".
- In python programming, polymorphism can be implemented in two ways they are:
 - 1) Method overriding
 - 1) Constructor overriding

Method overriding in python:-

→ Method overriding = Method Heading is same + Method Body is Different
(OR)

→ The process of re-defining the original method of base class into various derived classes for performing different operations is called method overriding.

→ To use method overriding in python program we must apply inheritance principle.

→ Method overriding used for implementing polymorphism principle.

(POLYMORPHISM ← -- METHOD OVERRIDING ← -- INHERITANCE
← -- CLASS AND OBJECTS)

Programming for demonstrating non-inheritance / Nonpolymorphism

```
class C1:
    def disp1(self):
        print("C1 -- disp1()")
```

ex1)

```
class C2:
    def disp2(self):
        print("C2 --- disp2()")
```

main program -- we created 4 objects

```
o1 = C1()
o2 = C2()
o1.disp1()
o2.disp2()
```

ex2) me).

```
o2 = C2()
o2.disp1()
o2.disp2()
```

 } — we created 2 objects
→ if we class C3

demonstrating polymorphism

ex1). Class C1:

```
def disp(self): # original method
    print("C1 --- disp()")
```

Class C2(C1):

```
def disp(self): # overridden method
    print("C2 --- disp()")
    super().disp()
```

→ C3 ki class Banao

main program - - - we created 2 objects..

print("w.r.t C3 class")

o3 = C3()

o3.disp()

ex2). sab same hai (super method mein), C3 ki class #; C1.disp(self)
C2.disp(self)

ex3). Class C1:

```
def __init__(self): # original constructor
    print("C1 --- default constructor")
```

Class C2(C1):

```
def __init__(self): # overridden constructor
    print("C2 --- default constructor")
    super().__init__()
```

Class C3(C2):

```
def __init__(self): # overridden constructor
    print("C3 --- default constructor")
    super().__init__()
```

main program we created 2 objects.

print("w.r.t C3 class")

o3 = C3 # Object creation -- calls constructor

ex4). Ab class C3 #; super hatake C2.__init__(self) } super hata dena
C1.__init__(self) } wahi
Back sub same.

ex5). Class Circle:

```
def area(self): # original method
    self.r = float(input("Enter radius:"))
    self.ac = 3.14 * self.r ** 2
    print("Area of Circle: {}".format(self.ac))
```

Class Square(Circle):

```
def area(self):
    self.s = float(input("Enter side:"))
    self.sa = self.s ** 2
    print("Area of Square: {}".format(self.sa))
    print(" — — — — — ")
    super().area()
```

Class Rect(Square):

```
def area(self):
    self.l = float(input("Enter length:"))
    self.b = float(input("Enter breadth:"))
    self.ra = self.l * self.b
    print("Area of Rect: {}".format(self.ra))
    print(" — — — — — ")
    super().area()
```

#main program

```
r = Rect()
r.area()
```

ex6). class circle- as it is,

class square- as it is (base super) new.

Class Rect(Circle, square) both.

STZ main program # bas r = Rect

print weke baad

square.area(self)

ex7) sab jagah def __init__(self) sthe super ni (sepe

1 jega super = super().__init__()

square.__init__(self)

ex 8) → class circle same self, r,
 → square same self, s.
 → class Rect (circle, square):
 ~~def~~ def area (self, l, b): # overridden method.
 self.ra = l*b
 print ("Area of Rect: {}".format (self.ra))
 print (" - - - - - ")
 super().area (float (input ("Enter radius:")),
 float (input ("Enter side:")))
 square.area (self, float (input ("Enter side:")))

 # Main Program
 r = Rect ()
 r.area (float (input ("Enter length")), float (input ("breadth")))

ex 9) def __init__ (self, r):
 super().__init__()
 square.__init__()
 r = Rect (float (input ("Enter radius:")), float (input ("Enter side:")))

 # Number of Approaches to call original methods/Constructors from overridden methods/Constructors :-
 X
 → we have two approaches to call original method/Constructors of base class from overridden method/Constructors of derived class. They are:
 1) By using super ()
 2) By using class Name
1) By using super ():
 → super () is one of the pre-defined function, which is used for calling super class original method/Constructor from overridden method/Constructors of derived class.
Syntax 1) super ().methodname (list of values)
 super ().methodname ()
Syntax 2) super ().__init__ (list of values)
 super ().__init__ ()

→ with `super()` we are able to call only immediate base class method/constructor but unable to call specified method/constructor of base class. to do this we must use class name approach.

2) By using class Name:-

→ By using class Name approach, we can call any base class method/constructor name from the context of derived class method/constructor name.

syntax1) `ClassName.method name(self, list of values)`
`Class Name.method name(self)`

syntax2) `Class Name -- init-- (self, list of values)`
`Class Name -- init-- (self)`

```
In [14]: #InhProg1
class C1:
    def getA(self):
        self.a=100
class C2(C1):
    def getB(self):
        self.b=200
    def addop(self):
        self.c=self.a+self.b
        print("Sum({}, {})={}".format(self.a,self.b,self.c))
#Main Program
o2=C2()
print(o2.__dict__)
o2.getB()
print(o2.__dict__)
o2.getA()
print(o2.__dict__)
o2.addop()
```

```
{}
{'b': 200}
{'b': 200, 'a': 100}
Sum(100,200)=300
```

```
In [15]: #InhProg2
class C1:
    def disp1(self):
        print("C1--disp1()")
class C2(C1):
    def disp2(self):
        print("C2--disp2()")
#Main Program
o2=C2()
o2.disp1()
o2.disp2()
```

```
C1--disp1()
C2--disp2()
```

```
In [18]: #InhProg3
class University:
    def getUnivDet(self):
        self.uname=input("Enter Univesity Name:")
        self.uloc=input("Enter University Location:")
    def dispUnivDet(self):
        print("-"*50)
        print("University Details")
        print("-" * 50)
        print("University Name:{}".format(self.uname))
        print("University Location:{}".format(self.uloc))
        print("-" * 50)
class College(University):
    def getCollDet(self):
        self.cname=input("Enter College Name:")
        self.cloc=input("Enter College Location:")
        self.getUnivDet()# Calling Base Class Method from Derived class Method
    def dispCollDet(self):
        self.dispUnivDet()# Calling Base Class Method from Derived Class Method
        print("-" * 50)
        print("College Details")
        print("-" * 50)
        print("College Name:{}".format(self.cname))
        print("College Location:{}".format(self.cloc))
        print("-" * 50)
class Student(College):
    def getStudDet(self):
        self.sno=int(input("Enter Student Number:"))
        self.sname=input("Enter Student Name:")
        self.crs=input("Enter Student Course:")
        self.getCollDet() # Calling Base Class Method from Derived class Method
    def dispStudDet(self):
        self.dispCollDet() # Calling Base Class Method from Derived class Method
        print("-"*50)
        print("Student Details")
        print("-" * 50)
        print("Student Number:{}".format(self.sno))
        print("Student Name:{}".format(self.sname))
        print("Student Course:{}".format(self.crs))
        print("-" * 50)
```



```
#Main Program
s=Student()
s.getStudDet()
s.dispStudDet()
```

University Details

University Name:pune university
University Location:pune

College Details

College Name:sitrc
College Location:nashik

Student Details

Student Number:1
Student Name:arif
Student Course:engineering

```
In [7]: #InhProg4
class Circle:
    def carea(self):
        self.r=float(input("Enter Radius:"))
        self.ac=3.14*self.r**2
        print("Area of Circle=",self.ac)
class Square(Circle):
    def sarea(self):
        self.s=float(input("Enter Side:"))
        self.sa=self.s**2
        print("Area of Square=",self.sa)
class Rect(Square):
    def rarea(self):
        self.L=float(input("Enter Length:"))
        self.B = float(input("Enter Length:"))
        self.ra=self.L*self.B
        print("Area of Rect=",self.ra)

#Main Program
r=Rect()
r.rarea()
r.sarea()
r.carea()
```

Area of Rect= 40.0
Area of Square= 49.0
Area of Circle= 78.5

```
In [8]: #poly ex1:-
class Circle:
    def area(self): # Original Method
        self.r = float(input("Enter Radius:"))
        self.ac = 3.14 * self.r ** 2
        print("Area of Circle=", self.ac)
class Square(Circle):
    def area(self): #Overridden Method
        self.s = float(input("Enter Side:"))
```

```

        self.sa = self.s ** 2
        print("Area of Square=", self.sa)
        super().area()
class Rect(Square):
    def area(self): #Overridden Method
        self.L = float(input("Enter Length:"))
        self.B = float(input("Enter Length:"))
        self.ra = self.L * self.B
        print("Area of Rect=", self.ra)
        super().area()

#Main program
r=Rect()
r.area()

```

Area of Rect= 20.0
 Area of Square= 64.0
 Area of Circle= 78.5

In [9]:

```

#PolyEx2
class Circle:
    def area(self): # Original Method
        self.r = float(input("Enter Radius:"))
        self.ac = 3.14 * self.r ** 2
        print("Area of Circle=", self.ac)
class Square:
    def area(self): #Original Method
        self.s = float(input("Enter Side:"))
        self.sa = self.s ** 2
        print("Area of Square=", self.sa)
class Rect(Square,Circle):
    def area(self):#Overridden Method
        self.L = float(input("Enter Length:"))
        self.B = float(input("Enter Length:"))
        self.ra = self.L * self.B
        print("Area of Rect=", self.ra)
        super().area()
        Circle.area(self)

#Main program
r=Rect()
r.area()

```

Area of Rect= 35.0
 Area of Square= 1.0
 Area of Circle= 78.5

In [10]:

```

#PolyEx3
class Circle:
    def __init__(self): # Original Constructor
        self.r = float(input("Enter Radius:"))
        self.ac = 3.14 * self.r ** 2
        print("Area of Circle=", self.ac)
class Square(Circle):
    def __init__(self):# Overridden Constructor
        self.s = float(input("Enter Side:"))
        self.sa = self.s ** 2
        print("Area of Square=", self.sa)
        super().__init__()

class Rect(Square):

```

```

def __init__(self):# Overridden Constructor
    self.L = float(input("Enter Length:"))
    self.B = float(input("Enter Breadth:"))
    self.ra = self.L * self.B
    print("Area of Rect=", self.ra)
    super().__init__()
#main Program
r=Rect()# Object Creation

```

Area of Rect= 40.0

Area of Square= 49.0

Area of Circle= 50.24

```

In [11]: #PolyEx4
class Circle:
    def __init__(self): # Original Constructor
        self.r = float(input("Enter Radius:"))
        self.ac = 3.14 * self.r ** 2
        print("Area of Circle=", self.ac)
class Square(Circle):
    def __init__(self):# Overridden Constructor
        self.s = float(input("Enter Side:"))
        self.sa = self.s ** 2
        print("Area of Square=", self.sa)
class Rect(Square):
    def __init__(self):# Overridden Constructor
        self.L = float(input("Enter Length:"))
        self.B = float(input("Enter Breadth:"))
        self.ra = self.L * self.B
        print("Area of Rect=", self.ra)
        Circle.__init__(self)
        Square.__init__(self)

#main Program
r=Rect()# Object Creation

```

Area of Rect= 20.0

Area of Circle= 153.86

Area of Square= 64.0

```

In [13]: #PolyEx5
class Circle:
    def area(self,r): # Original Method
        self.ac = 3.14 * r ** 2
        print("Area of Circle=", self.ac)
class Square:
    def area(self,s): #Original Method
        self.sa = s ** 2
        print("Area of Square=", self.sa)
class Rect(Square,Circle):
    def area(self,l,b):#Overridden Method
        self.ra = l*b
        print("Area of Rect=", self.ra)
        super().area(float(input("Enter Side:")))
        Circle.area(self,float(input("Enter Radius:")))

#Main program
r=Rect()
r.area(float(input("Enter Length:")),float(input("Enter Breadth:")))

```

Area of Rect= 20.0

Area of Square= 1.0

Area of Circle= 12.56

In []: